

KF 与 EKF 的朴素推导笔记

从“怎么把预测和观测揉在一起”到“怎么真正写成代码”

2026 年 3 月 29 日

目录

1	先说人话：这件事到底在解决什么问题	3
2	贯穿全文的例子：跟踪一辆小车	3
2.1	线性版本：GPS 直接测位置	3
2.2	非线性版本：路边雷达只测距离	4
3	符号表：后面反复出现的量	5
4	KF：线性系统下的最优融合	5
4.1	第一步：把问题写成状态空间模型	5
4.2	第二步：先做预测	6
4.2.1	状态均值怎么预测	6
4.2.2	误差协方差怎么预测	6
4.2.3	顺手预测一下观测	7
4.3	第三步：更新的结构其实先天就长这样	7
4.4	第四步：Kalman 增益为什么是这个公式	10
4.5	KF 全流程汇总	13
4.5.1	预测	14
4.5.2	更新	14
5	一个能手算的 KF 数值例子	14
5.1	设定	15
5.2	预测	15
5.3	更新	15
5.4	这组数字说明了什么	16

6 EKF: KF 的结构不变, 只是把非线性局部拉直	16
6.1 KF 为什么不能直接处理非线性	16
6.2 EKF 的核心近似	16
6.3 EKF 预测	17
6.4 EKF 更新	17
6.5 这到底和 KF 差在哪	17
7 一个能看懂 EKF 的数值例子	18
7.1 场景	18
7.2 先算预测观测	18
7.3 算雅可比	18
7.4 算增益并更新	19
7.5 直觉解释	19
8 KF 与 EKF 的差异, 压缩成四句话	19
9 从公式走到代码: 真正有用的落地细节	19
9.1 Q、R、P0 分别该怎么理解	19
9.1.1 R: 传感器有多吵	19
9.1.2 Q: 模型有多不靠谱	20
9.1.3 P0: 初始时你到底有多心虚	20
9.2 工程上强烈建议的实现细节	20
9.2.1 1. 协方差更新优先用 Joseph 形式	20
9.2.2 2. 每一步后强制对称化	20
9.2.3 3. 别直接求逆, 改解线性方程	20
9.2.4 4. EKF 的雅可比必须在正确工作点计算	21
9.2.5 5. 有角度就做角度归一化	21
9.2.6 6. 观测缺失时不要硬更新	21
9.3 什么时候 EKF 容易失效	21
10 最小可执行伪代码	21
10.1 KF	21
10.2 EKF	22
11 如果只记住一页内容	22
12 附: 从学习视角再看一眼 KF / EKF	23

阅读说明 这份 ‘main’ PDF 保留主线叙述。需要更细的逐步推导时, 正文会给出跳到子 PDF 的入口; 子 PDF 放在 ‘derivations/’ 目录下。

1 先说人话：这件事到底在解决什么问题

你每一秒都在做两件事：

1. 用模型猜一下下一秒会发生什么；
2. 看一下传感器，再决定要不要改口。

问题是，这两边都不可靠。

- 模型不可能完美，所以“预测”会飘；
- 传感器有噪声，所以“观测”也会抖。

Kalman Filter (KF) 干的事，可以先完全不看公式地理解成一句话：

在每一步，把“模型预测”和“新观测”按可信度做最优加权融合。

其中“可信度”不是拍脑袋写死的超参数，而是通过协方差自动算出来的。

Extended Kalman Filter (EKF) 则是 KF 的自然升级版：

如果系统或传感器是非线性的，就先在当前点附近用切线近似成线性的，再照着 KF 的套路走一遍。

所以 KF 和 EKF 的核心结构并不神秘。真正的难点只有两个：

1. 你怎么建模状态、观测、噪声；
2. 你怎么理解并设置 Q, R, P_0 这些协方差。

2 贯穿全文的例子：跟踪一辆小车

我们一直用同一个场景。

2.1 线性版本：GPS 直接测位置

一辆小车沿直线运动。我们想估计它的：

$$\mathbf{x}_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix}$$

其中 p_t 是位置， v_t 是速度。

假设采样周期是 Δt ，最朴素的运动模型是：

$$\begin{aligned} p_t &= p_{t-1} + \Delta t v_{t-1} + w_t^{(p)}, \\ v_t &= v_{t-1} + w_t^{(v)}. \end{aligned}$$

这里的 $w_t^{(p)}$ 和 $w_t^{(v)}$ 分别表示“位置方程没写准多少”和“速度方程没写准多少”。把它们打包起来，就是过程噪声向量

$$\mathbf{w}_t = \begin{bmatrix} w_t^{(p)} \\ w_t^{(v)} \end{bmatrix}.$$

这时状态向量的第一维是位置，第二维是速度；过程噪声向量的第一维对应位置方程的建模误差，第二维对应速度方程的建模误差。于是把两条标量方程打包后，写成矩阵就是：

$$\mathbf{x}_t = \mathbf{F} \mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}.$$

如果 GPS 直接测位置，那么它每一拍给你的不是“位置和速度一起的完整状态”，而只是一个标量位置读数。所以观测值 y_t 等于状态里的第一项 p_t 再加上测量噪声 v_t 。这里的观测矩阵只负责“从状态里拿出被传感器看到的那一项”：

$$\mathbf{H} \mathbf{x}_t = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p_t \\ v_t \end{bmatrix} = p_t.$$

也就是说，这个例子中的 GPS 只直接看位置，不直接看速度。于是观测模型写成矩阵就是：

$$y_t = \mathbf{H} \mathbf{x}_t + v_t, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

这就是线性系统，可以直接用 KF。

2.2 非线性版本：路边雷达只测距离

现在把 GPS 换成雷达。雷达不直接告诉你位置，它只告诉你车到雷达站的距离。

如果雷达站在原点，而小车二维位置是

$$\mathbf{x}_t = \begin{bmatrix} p_{x,t} \\ p_{y,t} \\ v_{x,t} \\ v_{y,t} \end{bmatrix},$$

那么观测可能是

$$y_t = h(\mathbf{x}_t) + v_t = \sqrt{p_{x,t}^2 + p_{y,t}^2} + v_t.$$

这里的 $h(\cdot)$ 已经不是直线，而是开方函数。于是标准 KF 不能直接上，得改用 EKF。

3 符号表：后面反复出现的量

符号	含义
$\mathbf{x}_t$	时刻 t 的真实状态，比如位置、速度、偏置、参数等
$\hat{\mathbf{x}}_{t t-1}$	用时刻 $t-1$ 的信息，对时刻 t 做出的预测
$\hat{\mathbf{x}}_{t t}$	融合了时刻 t 观测之后的最新估计
$\mathbf{P}_{t t-1}$	预测误差协方差，描述“我对预测有多没底”
$\mathbf{P}_{t t}$	更新后的误差协方差
$\mathbf{Q}$	过程噪声协方差，描述模型本身有多不可靠
$\mathbf{R}$	观测噪声协方差，描述传感器有多吵
$\mathbf{K}_t$	Kalman 增益，决定这一拍到底更信预测还是更信观测
$\mathbf{F}$	状态转移矩阵，描述状态怎么随时间推进
$\mathbf{H}$	观测矩阵，描述状态如何映射到观测
$\mathbf{y}_t$	观测值
$\tilde{\mathbf{y}}_t$	残差/创新，即“实际观测 - 预测观测”
$\mathbf{S}_t$	残差协方差，描述创新本身有多大不确定性

4 KF：线性系统下的最优融合

4.1 第一步：把问题写成状态空间模型

KF 的标准前提是：

$$\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}),$$

$$\mathbf{y}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R}).$$

这里 $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ 的意思是：过程噪声是一个零均值高斯向量，协方差是 $\mathbf{Q}$ 。拆开说就是：

$$\mathbb{E}[\mathbf{w}_t] = \mathbf{0}, \quad \text{Cov}(\mathbf{w}_t) = \mathbf{Q}.$$

又因为这里均值本来就是 0，所以协方差定义

$$\text{Cov}(\mathbf{w}_t) = \mathbb{E}[(\mathbf{w}_t - \mathbf{0})(\mathbf{w}_t - \mathbf{0})^\top] = \mathbb{E}[\mathbf{w}_t \mathbf{w}_t^\top] = \mathbf{Q}.$$

后面一看到 $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ ，本质上就是在调用这两条：一条给你零均值，一条给你噪声二阶矩等于 $\mathbf{Q}$ 。

直觉上：

- $\mathbf{F}\mathbf{x}_{t-1}$ 是“如果世界很乖，我预测它会这样走”；

- $\mathbf{w}_t$ 是“世界并不乖，所以真实状态会偏离模型”；
- $\mathbf{H}\mathbf{x}_t$ 是“如果传感器理想，它本该看到什么”；
- $\mathbf{v}_t$ 是“传感器实际还会抖一下”。

4.2 第二步：先做预测

4.2.1 状态均值怎么预测

这一步不该只靠“最自然”来糊过去。先把“预测均值”到底是什么意思说清楚。

到时刻 $t-1$ 为止，我们手里已经有全部旧信息，所以 $\hat{\mathbf{x}}_{t-1|t-1}$ 本质上是在这些旧信息条件下，对真实状态 $\mathbf{x}_{t-1}$ 的条件期望。

同理，时刻 t 的预测均值定义成：

$$\hat{\mathbf{x}}_{t|t-1} = \mathbb{E}[\mathbf{x}_t \mid \mathcal{Y}_{1:t-1}],$$

其中 $\mathcal{Y}_{1:t-1}$ 表示“截至 $t-1$ 时刻我们已经看到的全部观测信息”。

enter 把状态方程 $\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t$ 代进这个条件期望。

$$\hat{\mathbf{x}}_{t|t-1} = \mathbb{E}[\mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t \mid \mathcal{Y}_{1:t-1}].$$

rw 因为 $\mathbf{F}$ 是确定矩阵，所以可以从期望外提出来：

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F} \mathbb{E}[\mathbf{x}_{t-1} \mid \mathcal{Y}_{1:t-1}] + \mathbb{E}[\mathbf{w}_t \mid \mathcal{Y}_{1:t-1}].$$

rw 现在处理第二项。过程噪声 $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ ，而且默认它和过去观测独立，所以即便在“已经看到过去观测”的条件下，它的均值仍然是 0：

$$\mathbb{E}[\mathbf{w}_t \mid \mathcal{Y}_{1:t-1}] = \mathbf{0}.$$

done 再把 $\mathbb{E}[\mathbf{x}_{t-1} \mid \mathcal{Y}_{1:t-1}]$ 认成上一拍更新后的估计 $\hat{\mathbf{x}}_{t-1|t-1}$ ，就得到

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1|t-1}.$$

这条式子的真正意思是：先把上一拍最好的状态估计沿动力学推进，再利用“过程噪声零均值”把纯随机漂移的均值贡献消掉。

4.2.2 误差协方差怎么预测

定义预测误差

$$\mathbf{e}_{t|t-1} = \mathbf{x}_t - \hat{\mathbf{x}}_{t|t-1},$$

代入状态方程后有

$$\mathbf{e}_{t|t-1} = \mathbf{F}\mathbf{e}_{t-1|t-1} + \mathbf{w}_t.$$

所以预测协方差的主结果是：

$$\mathbf{P}_{t|t-1} = \mathbb{E}[\mathbf{e}_{t|t-1}\mathbf{e}_{t|t-1}^\top] = \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^\top + \mathbf{Q}.$$

这条式子的意思很朴素：下一拍的不确定性，等于“旧的不确定性被动力学传过去”再加上“这一拍新增的过程噪声”。

如果你想看这里为什么会冒出交叉项、为什么交叉项最后为 0、以及 $\mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ 到底是怎么落到 $\mathbb{E}[\mathbf{w}_t\mathbf{w}_t^\top] = \mathbf{Q}$ 上的，细节推导见：

[误差协方差预测：逐步推导](#) (derivations/covariance_prediction.pdf)

4.2.3 顺手预测一下观测

因为后面要和真实观测比，所以还要算：

$$\hat{\mathbf{y}}_{t|t-1} = \mathbf{H}\hat{\mathbf{x}}_{t|t-1}.$$

并定义创新（也叫残差）：

$$\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}.$$

创新就是一句大白话：“现实和我刚才的预测差了多少。”

到这里为止，我们只做完了“先猜一下下一拍会发生什么”。但滤波真正有价值的地方，不是猜，而是**观测来了之后，怎么系统地改口**。

所以接下来其实是两个连续问题：

1. 更新公式应该长什么样；
2. 这个更新公式里唯一的自由度“那个负责把残差变成修正量的矩阵”应该怎么选。

没有第一个问题，就谈不上第二个问题；没有第二个问题，后面那个增益矩阵的公式就会看起来像天上掉下来的。

4.3 第三步：更新的结构其实先天就长这样

这一节不先抛结论，而是从“更新时你手里到底有什么”一步步往下收。

第一步：更新时，手里其实只有两类信息 到更新开始这一刻，你并不知道真实状态 $\mathbf{x}_t$ ，你手里只有：

1. 刚刚算出来的预测 $\hat{\mathbf{x}}_{t|t-1}$ ；
2. 传感器刚给你的新观测 $\mathbf{y}_t$ 。

所以更新后的估计不可能凭空依赖别的东西，只能写成某个函数：

$$\hat{\mathbf{x}}_{t|t} = \Phi(\hat{\mathbf{x}}_{t|t-1}, \mathbf{y}_t).$$

第二步：真正有用的不是观测本身，而是“观测比预测多说了什么” 如果直接把 $\mathbf{y}_t$ 塞进更新公式，你很难看出“这次观测到底带来了多少新信息”。更自然的做法，是先拿观测和预测观测做比较。

我们前面已经定义了预测观测

$$\hat{\mathbf{y}}_{t|t-1} = \mathbf{H}\hat{\mathbf{x}}_{t|t-1},$$

以及创新

$$\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}.$$

这里要补一句很关键的话：在时刻 t 的更新阶段， $\hat{\mathbf{x}}_{t|t-1}$ 已经由预测步骤算完了，所以 $\hat{\mathbf{y}}_{t|t-1}$ 也是一个已知量，不再是未知变量。

因此，对这一拍来说， $\mathbf{y}_t$ 和 $\tilde{\mathbf{y}}_t$ 其实是等价的：知道其中一个，就能立刻算出另一个。

$$\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}, \quad \mathbf{y}_t = \tilde{\mathbf{y}}_t + \hat{\mathbf{y}}_{t|t-1}.$$

也就是说，把更新公式写成依赖 $\mathbf{y}_t$ 还是依赖 $\tilde{\mathbf{y}}_t$ ，在信息量上并没有损失。

只是从表达上说， $\tilde{\mathbf{y}}_t$ 更好，因为它直接表示“这次观测比预测多说了多少”。所以我们更愿意用创新来写更新公式。

所以更新公式更应该写成：

$$\hat{\mathbf{x}}_{t|t} = \Psi(\hat{\mathbf{x}}_{t|t-1}, \tilde{\mathbf{y}}_t).$$

第三步：如果创新为 0，就不该改口 这是最基本的约束。如果 $\tilde{\mathbf{y}}_t = \mathbf{0}$ ，说明观测和预测完全一致，那么更新后的估计就应该等于预测本身：

$$\tilde{\mathbf{y}}_t = \mathbf{0} \quad \implies \quad \hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1}.$$

这会逼着更新公式长成“旧预测 + 一个由创新决定的修正项”：

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{c}_t, \quad \mathbf{c}_t = g(\tilde{\mathbf{y}}_t), \quad g(\mathbf{0}) = \mathbf{0}.$$

第四步：把这个更新律在零创新附近做一阶线性展开 这时可以换一个更统一的视角来看。既然在时刻 t ， $\hat{\mathbf{x}}_{t|t-1}$ 已经是已知量，那么更新律本质上就是“创新 $\tilde{\mathbf{y}}_t$ 进来，吐出更新后估计”的一个函数。

把 $\hat{\mathbf{x}}_{t|t-1}$ 视为常量后，可以把它记成

$$\hat{\mathbf{x}}_{t|t} = \varphi_t(\tilde{\mathbf{y}}_t).$$

而上一步已经告诉我们：

$$\varphi_t(\mathbf{0}) = \hat{\mathbf{x}}_{t|t-1}.$$

所以如果在 $\tilde{\mathbf{y}}_t = \mathbf{0}$ 附近做一阶展开，就有

$$\varphi_t(\tilde{\mathbf{y}}_t) \approx \varphi_t(\mathbf{0}) + \left. \frac{\partial \varphi_t}{\partial \tilde{\mathbf{y}}} \right|_{\tilde{\mathbf{y}}=\mathbf{0}} \tilde{\mathbf{y}}_t.$$

把 $\varphi_t(\mathbf{0}) = \hat{\mathbf{x}}_{t|t-1}$ 代进去，再把那个雅可比矩阵记作 $\mathbf{G}_t$ ，就得到

$$\hat{\mathbf{x}}_{t|t} \approx \hat{\mathbf{x}}_{t|t-1} + \mathbf{G}_t \tilde{\mathbf{y}}_t.$$

现在才把这个待定矩阵 $\mathbf{G}_t$ 正式记作 $\mathbf{K}_t$ ：

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t \tilde{\mathbf{y}}_t.$$

这个视角和 EKF 很统一：都是“先有一个一般更新律，再在当前点附近抓它的一阶线性部分”。只是在线性 KF 里，我们最后会进一步证明：最优更新本来就落在这种线性族里，所以这里不只是拍脑袋近似，而是恰好抓住了正确的结构。

这里的 $\mathbf{K}_t$ 叫 Kalman 增益。它不是一个神秘常数，而是一个“把观测残差翻译成状态修正量”的线性映射。

如果状态维度是 n ，观测维度是 m ，那么

$$\mathbf{x}_t \in \mathbb{R}^n, \quad \tilde{\mathbf{y}}_t \in \mathbb{R}^m, \quad \mathbf{K}_t \in \mathbb{R}^{n \times m}.$$

也就是说， $\mathbf{K}_t$ 的每一列都在回答一个问题：“某一维观测差了这么多，我该把这种差异怎么分摊到各个状态维度上？”

在本文那辆小车的例子中，

$$\mathbf{x}_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix}, \quad y_t \in \mathbb{R},$$

所以创新 $\tilde{y}_t$ 是一个标量，而 $\mathbf{K}_t$ 是一个二维列向量：

$$\mathbf{K}_t = \begin{bmatrix} k_p \\ k_v \end{bmatrix}.$$

于是更新式其实是在说：

$$\hat{p}_{t|t} = \hat{p}_{t|t-1} + k_p \tilde{y}_t, \quad \hat{v}_{t|t} = \hat{v}_{t|t-1} + k_v \tilde{y}_t.$$

这句话很关键：虽然传感器这次只直接看到了位置，但位置残差依然能顺带修正速度；修正多少，由 $\mathbf{K}_t$ 自动决定。

所以 Kalman 增益本质上就是：

把“观测误差”翻译成“状态修正量”的矩阵。

如果你想看这条更新结构为什么几乎是被问题形式逼出来的，以及为什么 K_t 在标量观测时会退化成“给每个状态分配一个修正系数”的列向量，细节推导见：

[更新结构与 Kalman 增益含义：逐步说明](#) (derivations/update_structure.pdf)

4.4 第四步：Kalman 增益为什么是这个公式

上一节其实只做了一半。它告诉我们：合理的更新族应该写成

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + K_t \tilde{\mathbf{y}}_t.$$

但这时 K_t 还完全没定。也就是说，更新结构已经确定了，但“这次残差到底该信多少”还没定。

这一节真正要回答的，不是一个纯代数问题，而是三个很朴素的问题：

1. 这次残差 $\tilde{\mathbf{y}}_t$ 里，哪些部分真的是“预测错了”；
2. 哪些部分只是“传感器自己在抖”；
3. 最后应该把这份残差按多大比例翻译成状态修正量。

第一步：先把创新拆开看，它并不等于“真相” 由观测方程

$$\mathbf{y}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t$$

和预测观测

$$\hat{\mathbf{y}}_{t|t-1} = \mathbf{H}\hat{\mathbf{x}}_{t|t-1}$$

可得

$$\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1} = \mathbf{H}(\mathbf{x}_t - \hat{\mathbf{x}}_{t|t-1}) + \mathbf{v}_t = \mathbf{H}\mathbf{e}_{t|t-1} + \mathbf{v}_t.$$

这句式子非常关键。更直白地说：传感器看到的，不是完整状态误差 $\mathbf{e}_{t|t-1}$ 本身，而只是它“能看见的那一部分”，然后再混上一层测量噪声。

这里的 $\mathbf{H}\mathbf{e}_{t|t-1}$ 就是“状态误差里被传感器看到的部分”。

例如在小车例子里，

$$\mathbf{x}_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix},$$

那么

$$\mathbf{H}\mathbf{e}_{t|t-1} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \text{位置误差} \\ \text{速度误差} \end{bmatrix} = \text{位置误差}.$$

也就是说，这个传感器这一拍只能直接看见“位置到底差了多少”，看不见完整的二维状态误差；然后它给出的这份“位置残差”里，还会再掺上一层测量噪声 $\mathbf{v}_t$ 。

所以残差大，并不自动意味着应该大改口。它可能真的是预测错了很多，也可能只是这一拍传感器特别吵。

第二步：于是自然会冒出一个新问题，这份残差本身有多不确定。既然创新 $\tilde{\mathbf{y}}_t$ 本身就混着预测误差和测量噪声，那我们必须先衡量：这一拍残差到底靠不靠谱。

最自然的量，就是它自己的协方差：

$$\mathbf{S}_t = \mathbb{E}[\tilde{\mathbf{y}}_t \tilde{\mathbf{y}}_t^\top].$$

把 $\tilde{\mathbf{y}}_t = \mathbf{H}\mathbf{e}_{t|t-1} + \mathbf{v}_t$ 代进去，并利用预测误差与当前测量噪声独立、且测量噪声零均值，就得到

$$\mathbf{S}_t = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \mathbf{R}.$$

这里的 $\mathbf{S}_t$ 叫**创新协方差**。它描述的不是“状态有多不确定”，而是“这一拍残差本身有多不确定”。

如果 $\mathbf{S}_t$ 很大，你就该先警惕一下：这次残差里可能掺了很多噪声，不能一股脑全信。

第三步：还要问，这份残差到底对哪些状态方向有用。不是每个状态维度都应该对残差做出同样反应。真正重要的是：预测误差和创新之间到底有多相关。

最自然的量，就是二者的互协方差：

$$\mathbb{E}[\mathbf{e}_{t|t-1} \tilde{\mathbf{y}}_t^\top].$$

把 $\tilde{\mathbf{y}}_t = \mathbf{H}\mathbf{e}_{t|t-1} + \mathbf{v}_t$ 代入：

$$\mathbb{E}[\mathbf{e}_{t|t-1} \tilde{\mathbf{y}}_t^\top] = \mathbb{E}[\mathbf{e}_{t|t-1} (\mathbf{H}\mathbf{e}_{t|t-1} + \mathbf{v}_t)^\top].$$

再展开：

$$\mathbb{E}[\mathbf{e}_{t|t-1} \tilde{\mathbf{y}}_t^\top] = \mathbb{E}[\mathbf{e}_{t|t-1} \mathbf{e}_{t|t-1}^\top] \mathbf{H}^\top + \mathbb{E}[\mathbf{e}_{t|t-1} \mathbf{v}_t^\top].$$

由于预测误差和当前测量噪声独立，而且 $\mathbb{E}[\mathbf{v}_t] = \mathbf{0}$ ，第二项为 0，所以

$$\mathbb{E}[\mathbf{e}_{t|t-1} \tilde{\mathbf{y}}_t^\top] = \mathbf{P}_{t|t-1} \mathbf{H}^\top.$$

这条式子的意思是：只有当某个状态方向和创新真的相关时，这份残差才有资格去修正那个状态方向。

第四步：最自然的线性修正，就是一个“回归系数矩阵”。上一节已经把更新结构固定成

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t \tilde{\mathbf{y}}_t.$$

等价地，更新后误差满足

$$\mathbf{e}_{t|t} = \mathbf{x}_t - \hat{\mathbf{x}}_{t|t} = \mathbf{e}_{t|t-1} - \mathbf{K}_t \tilde{\mathbf{y}}_t.$$

先说朴素想法。既然这一拍你已经把创新 $\tilde{\mathbf{y}}_t$ 拿来修正过一次了，那么一个“像样”的更新结果至少应该满足：修完之后，误差里不要还残留和这份创新同方向的、可继续利用的信息。

不然就说明这次观测里还有东西没用干净，你还可以顺着同一份残差继续往下修。

不过这句话目前还只是直觉，不是证明。真正严谨的做法，还是把优化目标明确写出来，然后对 $\mathbf{K}_t$ 求最小值。

现在直接走最短路线：先把要优化的量写出来，然后对 $\mathbf{K}_t$ 求最小值。

我们在线性更新族里真正想做的事，是选一个 $\mathbf{K}_t$ ，让更新后的均方误差最小：

$$J(\mathbf{K}_t) = \mathbb{E}[\|\mathbf{e}_{t|t}\|_2^2] = \text{tr}(\mathbf{P}_{t|t}).$$

最后这个等号也不是凭空来的。因为

$$\|\mathbf{e}_{t|t}\|_2^2 = \mathbf{e}_{t|t}^\top \mathbf{e}_{t|t} = \text{tr}(\mathbf{e}_{t|t} \mathbf{e}_{t|t}^\top),$$

所以

$$\mathbb{E}[\|\mathbf{e}_{t|t}\|_2^2] = \text{tr}(\mathbb{E}[\mathbf{e}_{t|t} \mathbf{e}_{t|t}^\top]) = \text{tr}(\mathbf{P}_{t|t}).$$

这里“迹和期望可交换”也不神秘。设随机矩阵 $\mathbf{A} = (A_{ij})$ ，那么

$$\text{tr}(\mathbb{E}[\mathbf{A}]) = \sum_i \mathbb{E}[A_{ii}] = \mathbb{E}\left[\sum_i A_{ii}\right] = \mathbb{E}[\text{tr}(\mathbf{A})].$$

中间这一步

$$\sum_i \mathbb{E}[A_{ii}] = \mathbb{E}\left[\sum_i A_{ii}\right]$$

用的只是期望的线性性：

$$\mathbb{E}[X_1 + \cdots + X_n] = \mathbb{E}[X_1] + \cdots + \mathbb{E}[X_n].$$

所以它本质上只是：**有限项求和可以靠期望的线性性移进移出。**

把 $\mathbf{e}_{t|t} = \mathbf{e}_{t|t-1} - \mathbf{K}_t \tilde{\mathbf{y}}_t$ 代进去：

$$J(\mathbf{K}_t) = \text{tr}\left(\mathbb{E}[(\mathbf{e}_{t|t-1} - \mathbf{K}_t \tilde{\mathbf{y}}_t)(\mathbf{e}_{t|t-1} - \mathbf{K}_t \tilde{\mathbf{y}}_t)^\top]\right).$$

把括号展开，再把前两步已经算出的

$$\mathbb{E}[\mathbf{e}_{t|t-1} \tilde{\mathbf{y}}_t^\top] = \mathbf{P}_{t|t-1} \mathbf{H}^\top, \quad \mathbb{E}[\tilde{\mathbf{y}}_t \tilde{\mathbf{y}}_t^\top] = \mathbf{S}_t$$

代进去，可得

$$\begin{aligned} J(\mathbf{K}_t) &= \text{tr}(\mathbf{P}_{t|t-1}) - 2 \text{tr}(\mathbf{K}_t \mathbf{H} \mathbf{P}_{t|t-1}) \\ &\quad + \text{tr}(\mathbf{K}_t \mathbf{S}_t \mathbf{K}_t^\top). \end{aligned}$$

现在直接对 $\mathbf{K}_t$ 求导。用到两条常见公式：

$$\frac{\partial}{\partial \mathbf{K}} \text{tr}(\mathbf{K} \mathbf{A}) = \mathbf{A}^\top, \quad \frac{\partial}{\partial \mathbf{K}} \text{tr}(\mathbf{K} \mathbf{S} \mathbf{K}^\top) = 2 \mathbf{K} \mathbf{S} \quad (\mathbf{S} = \mathbf{S}^\top).$$

于是

$$\frac{\partial J}{\partial \mathbf{K}_t} = -2 \mathbf{P}_{t|t-1} \mathbf{H}^\top + 2 \mathbf{K}_t \mathbf{S}_t.$$

令导数为 0，就得到最优条件：

$$\mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}^\top \mathbf{S}_t^{-1}.$$

这就是 Kalman 增益公式。

这条公式最朴素的读法

$$\mathbf{K}_t = \underbrace{\mathbf{P}_{t|t-1} \mathbf{H}^\top}_{\text{状态误差和创新有多相关}} \underbrace{\mathbf{S}_t^{-1}}_{\text{这份创新本身有多值得信}}$$

所以它根本不神秘，本质上就是一句大白话：

先看这份残差对状态误差有没有用，再看这份残差自己靠不靠谱，最后决定修多大。

在一维情形下，这条式子会直接退化成更熟悉的样子：

$$K_t = \frac{\text{Cov}(e_{t|t-1}, \tilde{y}_t)}{\text{Var}(\tilde{y}_t)}.$$

它其实就是一个线性回归系数：

- 分子大，说明创新 and 状态误差强相关，这份残差有用；
- 分母大，说明创新本身很吵，这份残差不太可靠；
- 二者一除，就得到“这次该修多大”。

补一句工程实现 到这里，Kalman 增益为什么长成这个公式，思想上已经结束了。

如果你在代码里还要继续维护更新后的协方差，常用的数值稳定写法是

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_{t|t-1} (\mathbf{I} - \mathbf{K}_t \mathbf{H})^\top + \mathbf{K}_t \mathbf{R} \mathbf{K}_t^\top.$$

这叫 Joseph 形式。它是一个**实现层面的稳健写法**，不是理解 $\mathbf{K}_t$ 来龙去脉的起点。

如果你想看上面这条链怎么一行一行算清，包括 $\mathbf{S}_t$ 怎么推出来、互协方差怎么推出来、以及 Joseph 形式为什么只是工程尾注，见：

[Kalman 增益：从创新相关性到最优修正](#) (derivations/kalman_gain.pdf)

4.5 KF 全流程汇总

这一节不是为了再推导一次，而是把前面的结果压成一页“可执行配方”。

读到这里如果已经把字母忘了，很正常。先把最常用的符号再翻回人话：

- $\hat{\mathbf{x}}_{t|t-1}$ ：看到第 t 拍观测之前，对当前状态的预测；
- $\mathbf{P}_{t|t-1}$ ：这份预测还有多不确定；
- $\hat{\mathbf{y}}_{t|t-1}$ ：如果当前预测没错，传感器大概应该读到什么；
- $\tilde{\mathbf{y}}_t$ ：真实观测和预测观测的差，也就是创新/残差；
- $\mathbf{S}_t$ ：这份残差本身有多不确定；
- $\mathbf{K}_t$ ：该把这份残差按多大力度翻译成状态修正量；
- $\mathbf{P}_{t|t}$ ：更新完之后，还剩多少不确定性。

4.5.1 预测

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1|t-1},$$

就是：先把上一拍最好的估计，沿着动力学模型往前推一拍。

$$\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^\top + \mathbf{Q},$$

就是：旧的不确定性被系统传过来，再加上这一拍新增的过程噪声。

$$\hat{\mathbf{y}}_{t|t-1} = \mathbf{H}\hat{\mathbf{x}}_{t|t-1},$$

就是：按当前预测来看，传感器这拍本来应该读到什么。

$$\mathbf{S}_t = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \mathbf{R}.$$

就是：这份“残差”自己到底有多靠谱，还是有多吵。

4.5.2 更新

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}^\top\mathbf{S}_t^{-1},$$

就是：先看残差对状态误差有没有用，再看这份残差自己靠不靠谱，最后决定修多大。

$$\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1},$$

就是：现实和刚才预测到底差了多少。

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t\tilde{\mathbf{y}}_t,$$

就是：用 Kalman 增益把残差翻译成状态修正量，加回预测值得到更新结果。

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\mathbf{P}_{t|t-1}(\mathbf{I} - \mathbf{K}_t\mathbf{H})^\top + \mathbf{K}_t\mathbf{R}\mathbf{K}_t^\top.$$

就是：更新之后，原先那份不确定性被压小了，但测量噪声本身也会留下影响。

5 一个能手算的 KF 数值例子

先故意用最简单的一维版本，把逻辑彻底看透。

5.1 设定

假设状态就是小车位置 x_t ，动态模型和观测模型分别是：

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, 1),$$

$$y_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, 4).$$

已知上一时刻更新后：

$$\hat{x}_{t-1|t-1} = 10, \quad P_{t-1|t-1} = 9.$$

这一时刻来了一个观测：

$$y_t = 13.$$

5.2 预测

因为这里 $F = 1$ ，所以

$$\hat{x}_{t|t-1} = 10.$$

$$P_{t|t-1} = 9 + 1 = 10.$$

5.3 更新

创新协方差：

$$S_t = P_{t|t-1} + R = 10 + 4 = 14.$$

Kalman 增益：

$$K_t = \frac{10}{14} \approx 0.714.$$

创新：

$$\tilde{y}_t = 13 - 10 = 3.$$

更新后的估计：

$$\hat{x}_{t|t} = 10 + 0.714 \times 3 \approx 12.14.$$

更新后的协方差：

$$P_{t|t} = (1 - 0.714) \times 10 \approx 2.86.$$

5.4 这组数字说明了什么

- 预测值是 10, 观测值是 13, 最后结果是 12.14, 没有完全信任何一边;
- 因为预测方差 10 比观测噪声 4 更大, 说明当前更该信观测, 所以 K_t 比较大;
- 更新后 P 从 10 下降到 2.86, 说明融合完这次观测之后, 我们心里更有数了。

6 EKF: KF 的结构不变, 只是把非线性局部拉直

6.1 KF 为什么不能直接处理非线性

如果系统变成:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}) + \mathbf{w}_t,$$

$$\mathbf{y}_t = h(\mathbf{x}_t) + \mathbf{v}_t,$$

那么麻烦在于:

- 经过 $f(\cdot)$ 之后, 高斯分布一般不再严格保持高斯;
- 协方差传播也不能直接写成 $\mathbf{F}\mathbf{P}\mathbf{F}^\top$;
- 观测映射 $h(\cdot)$ 也不能直接用一个固定矩阵 $\mathbf{H}$ 表示。

EKF 的处理思路很务实:

我不试图全局搞定这个弯函数, 而是只在“当前估计附近”把它一阶泰勒展开, 临时当成线性的。

6.2 EKF 的核心近似

在当前工作点附近,

$$f(\mathbf{x}) \approx f(\bar{\mathbf{x}}) + \mathbf{F}_t(\mathbf{x} - \bar{\mathbf{x}}), \quad \mathbf{F}_t = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}}},$$

$$h(\mathbf{x}) \approx h(\bar{\mathbf{x}}) + \mathbf{H}_t(\mathbf{x} - \bar{\mathbf{x}}), \quad \mathbf{H}_t = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}}}.$$

这里的 $\mathbf{F}_t$ 和 $\mathbf{H}_t$ 就是雅可比矩阵。它们的角色和线性 KF 里的 $\mathbf{F}, \mathbf{H}$ 完全一样, 只不过现在每一步都要重算。

6.3 EKF 预测

$$\hat{\mathbf{x}}_{t|t-1} = f(\hat{\mathbf{x}}_{t-1|t-1}),$$

$$\mathbf{F}_t = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{t-1|t-1}},$$

$$\mathbf{P}_{t|t-1} = \mathbf{F}_t \mathbf{P}_{t-1|t-1} \mathbf{F}_t^\top + \mathbf{Q}.$$

6.4 EKF 更新

$$\hat{\mathbf{y}}_{t|t-1} = h(\hat{\mathbf{x}}_{t|t-1}),$$

$$\mathbf{H}_t = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{t|t-1}},$$

$$\mathbf{S}_t = \mathbf{H}_t \mathbf{P}_{t|t-1} \mathbf{H}_t^\top + \mathbf{R},$$

$$\mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}_t^\top \mathbf{S}_t^{-1},$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t (\mathbf{y}_t - \hat{\mathbf{y}}_{t|t-1}),$$

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \mathbf{P}_{t|t-1} (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t)^\top + \mathbf{K}_t \mathbf{R} \mathbf{K}_t^\top.$$

6.5 这到底和 KF 差在哪

结构上几乎没差，真正的区别只有两条：

1. 状态预测不再是矩阵乘法，而是 $f(\cdot)$ ；
2. 传播协方差和做观测更新时，用的是当前点的雅可比 $\mathbf{F}_t, \mathbf{H}_t$ ，而不是固定矩阵。

所以 EKF 的一句话定义可以非常朴素：

KF 是“在线性世界里做最优融合”；EKF 是“先把局部世界近似成线性，再做近似最优融合”。

7 一个能看懂 EKF 的数值例子

7.1 场景

还追那辆车，但现在雷达只能测它到原点的距离。

假设当前预测状态是二维位置：

$$\hat{\mathbf{x}}_{t|t-1} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad \mathbf{P}_{t|t-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

观测模型是：

$$y_t = h(\mathbf{x}_t) + v_t = \sqrt{p_x^2 + p_y^2} + v_t, \quad R = 0.25.$$

实际观测到的距离是：

$$y_t = 4.7.$$

7.2 先算预测观测

$$\hat{y}_{t|t-1} = \sqrt{3^2 + 4^2} = 5.$$

创新：

$$\tilde{y}_t = 4.7 - 5 = -0.3.$$

7.3 算雅可比

对

$$h(\mathbf{x}) = \sqrt{p_x^2 + p_y^2}$$

求导：

$$\mathbf{H}_t = \begin{bmatrix} \frac{p_x}{\sqrt{p_x^2 + p_y^2}} & \frac{p_y}{\sqrt{p_x^2 + p_y^2}} \end{bmatrix}.$$

代入 $(p_x, p_y) = (3, 4)$ ，得到：

$$\mathbf{H}_t = \begin{bmatrix} 0.6 & 0.8 \end{bmatrix}.$$

7.4 算增益并更新

$$S_t = \mathbf{H}_t \mathbf{P}_{t|t-1} \mathbf{H}_t^\top + R = 0.6^2 + 0.8^2 + 0.25 = 1.25.$$

$$\mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}_t^\top S_t^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix} \frac{1}{1.25} = \begin{bmatrix} 0.48 \\ 0.64 \end{bmatrix}.$$

所以更新后：

$$\hat{\mathbf{x}}_{t|t} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} + \begin{bmatrix} 0.48 \\ 0.64 \end{bmatrix} (-0.3) = \begin{bmatrix} 2.856 \\ 3.808 \end{bmatrix}.$$

7.5 直觉解释

因为观测距离比预测距离更小，所以估计位置会往原点缩一点。
而且缩的方向不是乱缩，而是沿着雅可比

$$\begin{bmatrix} 0.6 & 0.8 \end{bmatrix}$$

指向的“距离增加最快方向”去修正。这就是 EKF 里雅可比真正的物理含义：

它告诉你，当前这个观测对各个状态维度到底敏感到什么程度。

8 KF 与 EKF 的差异，压缩成四句话

1. KF 假设状态转移和观测关系都是线性的；EKF 允许非线性。
2. KF 用固定的 $\mathbf{F}, \mathbf{H}$ ；EKF 每一步都要算雅可比 $\mathbf{F}_t, \mathbf{H}_t$ 。
3. KF 在线性高斯条件下是最优的；EKF 是局部线性化后的近似方法。
4. 非线性越强、初值越差、噪声越大，EKF 越容易发散。

9 从公式走到代码：真正有用的落地细节

9.1 Q、R、P0 分别该怎么理解

9.1.1 R：传感器有多吵

R 大，表示你对传感器不信任，更新时更偏向预测。

如果能单独采一段静态数据，直接估计传感器噪声方差，这是最靠谱的做法。

9.1.2 Q: 模型有多不靠谱

Q 大，表示你承认“真实系统经常偏离我写下来的动力学模型”，因此预测协方差会更大，滤波器也会更愿意接受新观测。

实践里最容易犯的错是：

- Q 设得过小：滤波器过度自信，遇到模型失配时改不回来；
- Q 设得过大：滤波器每一步都像失忆一样，输出抖。

9.1.3 P0: 初始时你到底有多心虚

P_0 是初始估计的不确定性。

- 如果初值很准， P_0 可以小一些；
- 如果初值只是拍脑袋给的， P_0 应该大一些。

它直接影响前几步 Kalman 增益的大小，所以别把它当成无关紧要的初始化细节。

9.2 工程上强烈建议的实现细节

9.2.1 1. 协方差更新优先用 Joseph 形式

不要偷懒只写

$$P = (I - KH)P.$$

推荐写成：

$$P = (I - KH)P(I - KH)^T + KRK^T.$$

原因：更稳，更不容易数值炸掉。

9.2.2 2. 每一步后强制对称化

实际代码中常见处理：

$$P \leftarrow \frac{P + P^T}{2}.$$

这是因为浮点误差会让本来应当对称的协方差出现微小偏斜。

9.2.3 3. 别直接求逆，改解线性方程

数学上写 S^{-1} 很方便，但代码里别真的调用矩阵求逆。优先用 Cholesky 分解或线性求解器。

原因：更稳，也更快。

9.2.4 4. EKF 的雅可比必须在正确工作点计算

最常见的选择是：

- 预测用的雅可比 F_t ，在 $\hat{x}_{t-1|t-1}$ 处算；
- 更新用的雅可比 H_t ，在 $\hat{x}_{t|t-1}$ 处算。

点选错了，EKF 可能直接漂掉。

9.2.5 5. 有角度就做角度归一化

如果观测里有航向角、方位角之类，创新不能直接做普通减法。比如 179° 和 -179° 的真实差不是 358° ，而是 2° 。

所以残差里出现角度时，必须先 wrap 到合法区间。

9.2.6 6. 观测缺失时不要硬更新

如果这一时刻没观测，或者观测被判为异常值，就只做预测，不做更新。

KF/EKF 天然支持这种模式。

9.3 什么时候 EKF 容易失效

1. 非线性太强，而一阶线性化太粗糙；
2. 初值离真实状态太远，导致切线方向本身就错；
3. 模型写错， Q 却又设得很小，滤波器被错误模型锁死；
4. 雅可比写错，这是最常见也最隐蔽的 bug；
5. 状态或协方差尺度差异极大，数值条件很差。

如果你遇到这些问题，常见替代方案包括：UKF、粒子滤波、迭代 EKF，或者先把状态做更合理的参数化与归一化。

10 最小可执行伪代码

10.1 KF

```
given x_hat, P
for each step t:
    # predict
    x_pred = F @ x_hat
```

```

P_pred = F @ P @ F.T + Q

y_pred = H @ x_pred
innov = y - y_pred
S = H @ P_pred @ H.T + R
K = P_pred @ H.T @ inv(S)

# update
x_hat = x_pred + K @ innov
I = eye(dim_x)
P = (I - K @ H) @ P_pred @ (I - K @ H).T + K @ R @ K.T

```

10.2 EKF

```

given x_hat, P
for each step t:
    # predict
    x_pred = f(x_hat)
    F = jacobian_f(at=x_hat)
    P_pred = F @ P @ F.T + Q

    # update
    y_pred = h(x_pred)
    H = jacobian_h(at=x_pred)
    innov = y - y_pred
    innov = normalize_angle_if_needed(innov)

    S = H @ P_pred @ H.T + R
    K = P_pred @ H.T @ inv(S)

    x_hat = x_pred + K @ innov
    I = eye(dim_x)
    P = (I - K @ H) @ P_pred @ (I - K @ H).T + K @ R @ K.T

```

11 如果只记住一页内容

1. KF 解决的是“模型预测”和“带噪观测”如何最优融合的问题。
2. 预测阶段靠模型推进均值和协方差；更新阶段靠创新和增益修正状态。

3. Kalman 增益不是魔法，它只是“不确定性驱动的自适应权重”。
4. EKF 不是另一套新宇宙，它只是把非线性系统在当前点局部线性化后，再走一遍 KF。
5. 真正决定效果的，往往不是公式会不会背，而是建模是否合理、雅可比是否正确，以及 Q 、 R 、 P_0 是否符合物理现实。

12 附：从学习视角再看一眼 KF / EKF

如果你更熟悉机器学习，可以把更新式

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t \tilde{\mathbf{y}}_t$$

看成一种“自适应学习率”的梯度式修正：

- $\tilde{\mathbf{y}}_t$ 类似误差；
- $\mathbf{K}_t$ 类似每个方向上的动态学习率；
- 但它不是手工调出来的，而是由 $\mathbf{P}$, $\mathbf{Q}$, $\mathbf{R}$ 自动推出来的。

这也是 KF / EKF 到今天仍然很有生命力的原因：

它不是“把公式背下来”，而是“把不确定性明确建模，再让不确定性决定每一步该学多大”。